Team notebook

August 3, 2026

Contents

1 Algorithms

1.1 Mo's algorithm on trees

```
/**
problems:
    - https://codeforces.com/gym/101161 problem E
*/
void flat(vector<vector<edge>> &g, vector<int> &a,
    vector<int> &le, vector<int> &ri, vector<int> &cost,
    int node, int pi, int &ts, int w) {

  cost[node] = w;
  le[node] = ts;
  a[ts] = node;
  ts++;
  for (auto e : g[node]) {
    if (e.to == pi) continue;
    flat(g, a, le, ri, cost, e.to, node, ts, e.w);
  }
  ri[node] = ts;
  a[ts] = node;
  ts++;
}

/**
 * Case when the cost is in the edges.
 * */
void compute_queries(vector<vector<edge>> &g) {
  // g is undirected
  int n = g.size();

  lca_tree.init(g, 0);

  vector<int> a(2 * n), le(n), ri(n), cost(n);
  // a: nodes in the flatten array
  // le: left id of the given node
  // ri: right id of the given node
  // cost: cost of the edge from the node to the parent
```

```
  int ts = 0; // timestamp
  flat(g, a, le, ri, cost, 0, -1, ts, 0);

  int q; cin >> q;
  vector<query> queries(q);
  for (int i = 0; i < q; i++) {
    int u, v;
    cin >> u >> v;
    u--; v--;
    int lca = lca_tree.query(u, v);
    if (le[u] > le[v])
      swap(u, v);
    queries[i].id = i;
    queries[i].lca = lca;
    queries[i].u = u;
    queries[i].v = v;
    if (lca == u) {
      queries[i].a = le[u] + 1;
      queries[i].b = le[v];
    } else {
      queries[i].a = ri[u];
      queries[i].b = le[v];
    }
  }
  solve_mo(queries, a, le, cost); // this is the usal
       algorithm
}
```

1.2 Mo's algorithm

```
const int MN = 5 * 100000 + 1;
const int SN = 708;

struct Query {
  int a, b, id;
  Query() {}
  Query(int x, int y, int i) : a(x), b(y), id(i) {}

  bool operator<(const Query &o) const {
    if (a / SN != o.a / SN) return a < o.a;
    return a / SN & 1 ? b < o.b : b > o.b;
  }
};

struct DS {
  DS() : {}

  void Insert(int x) {}

  void Erase(int x) {}

  long long Query() {}
};

Query s[MN];
int ans[MN];
DS active;

int main() {
  int n;
  cin >> n;
  vector<int> a(n);
  for (auto &i : a) cin >> i;

  int q;
  cin >> q;
  for (int i = 0; i < q; ++i) {
    int b, e;
    cin >> b >> e;
    b--;
    e--;
    s[i] = Query(b, e, i);
  }

  sort(s, s + q);

  int i = 0;
  int j = -1;
  for (int k = 0; k < (int)q; ++k) {
    int L = s[k].a;
    int R = s[k].b;

    while (j < R) active.Insert(a[++j]);
    while (j > R) active.Erase(a[j--]);
    while (i < L) active.Erase(a[i++]);
    while (i > L) active.Insert(a[--i]);

    ans[s[k].id] = active.Query();
  }

  for (int i = 0; i < q; ++i) {
    cout << ans[i] << endl;
  }

  return 0;
};
```

1.3 sliding window

```
/*
 * Given an array ARR and an integer K, the problem boils
      down to computing for each index i: min(ARR[i],
      ARR[i-1], ..., ARR[i-K+1]).
 * if mx == true, returns the maximun.
 * */

vector<int> sliding_window_minmax(vector<int> & ARR, int
     K, bool mx) {
  deque< pair<int, int> > window;
  vector<int> ans;
  for (int i = 0; i < ARR.size(); i++) {
    if (mx) {
      while (!window.empty() && window.back().first <=
           ARR[i])
        window.pop_back();
    } else {
      while (!window.empty() && window.back().first >=
           ARR[i])
        window.pop_back();
    }
    window.push_back(make_pair(ARR[i], i));

    while(window.front().second <= i - K)
      window.pop_front();

    ans.push_back(window.front().first);
  }
  return ans;
}
```

2 DP Optimizations

2.1 convex hull trick

```
struct line {
  long long m, b;
  line (long long a, long long c) : m(a), b(c) {}
  long long eval(long long x) {
    return m * x + b;
  }
};

long double inter(line a, line b) {
  long double den = a.m - b.m;
  long double num = b.b - a.b;
  return num / den;
}

/**
 *  min m_i * x_j + b_i, for all i.
 *     x_j <= x_{j + 1}
 *     m_i >= m_{j + 1}
 * */
struct ordered_cht {
  vector<line> ch;
  int idx; // id of last "best" in query
  ordered_cht() {
    idx = 0;
  }

  void insert_line(long long m, long long b) {
    line cur(m, b);
    // new line's slope is less than all the previous
    while (ch.size() > 1 &&
       (inter(cur, ch[ch.size() - 2]) >= inter(cur,
            ch[ch.size() - 1]))) {
      // f(x) is better in interval [inter(ch.back(),
           cur), inf)
      ch.pop_back();
    }

    ch.push_back(cur);
  }
```

```
  long long eval(long long x) { // minimum
    // current x is greater than all the previous x,
    // if that is not the case we can make binary search.
    idx = min<int>(idx, ch.size() - 1);
    while (idx + 1 < (int)ch.size() && ch[idx + 1].eval(x)
         <= ch[idx].eval(x))
      idx++;

    return ch[idx].eval(x);
  }
};


/**
 *  Dynammic convex hull trick
 * */

typedef long long int64;
typedef long double float128;

const int64 is_query = -(1LL<<62), inf = 1e18;

struct Line {
  int64 m, b;
  mutable function<const Line*()> succ;
  bool operator<(const Line& rhs) const {
    if (rhs.b != is_query) return m < rhs.m;
    const Line* s = succ();
    if (!s) return 0;
    int64 x = rhs.m;
    return b - s->b < (s->m - m) * x;
  }
};

struct HullDynamic : public multiset<Line> { // will
     maintain upper hull for maximum
  bool bad(iterator y) {
    auto z = next(y);
    if (y == begin()) {
      if (z == end()) return 0;
      return y->m == z->m && y->b <= z->b;
    }
    auto x = prev(y);
    if (z == end()) return y->m == x->m && y->b <= x->b;
    return (float128)(x->b - y->b)*(z->m - y->m) >=
         (float128)(y->b - z->b)*(y->m - x->m);
  }
  void insert_line(int64 m, int64 b) {
    auto y = insert({ m, b });
    y->succ = [=] { return next(y) == end() ? 0 :
         &*next(y); };
    if (bad(y)) { erase(y); return; }
    while (next(y) != end() && bad(next(y))) erase(next(y));
    while (y != begin() && bad(prev(y))) erase(prev(y));
  }

  int64 eval(int64 x) {
    auto l = *lower_bound((Line) { x, is_query });
    return l.m * x + l.b;
  }
};
```

2.2 divide and conquer

```
/**
 * recurrence:
 *    dp[k][i] = min dp[k-1][j] + c[i][j - 1], for all j >
      i;
 *
 * "comp" computes dp[k][i] for all i in O(n log n) (k is
      fixed)
 * */

void comp(int l, int r, int le, int re) {
  if (l > r) return;

  int mid = (l + r) >> 1;

  int best = max(mid + 1, le);
  dp[cur][mid] = dp[cur ^ 1][best] + cost(mid, best - 1);
  for (int i = best; i <= re; i++) {
    if (dp[cur][mid] > dp[cur ^ 1][i] + cost(mid, i - 1)) {
      best = i;
      dp[cur][mid] = dp[cur ^ 1][i] + cost(mid, i - 1);
    }
  }

  comp(l, mid - 1, le, best);
  comp(mid + 1, r, best, re);
}
```

2.3 dp on trees

```
/**
 * This trick is very useful when doing DP on trees,
      basically, you can save
 * the answer for each node as if it was the root of the
      tree. Partial results
 * are also stored in order to query subtrees (taking the
      root and exclude some
 * child).

 * */


struct edge {
  int to, p_id;
  edge (int a, int b) : to(a), p_id(b) {}
};

struct state {
  bool seen;
  long long missing;
  long long total;
  vector<long long> partial;

  state() { clear(); }

  void clear() {
    seen = false;
    missing = 0;
    total = 0;
    partial.clear();
  }
};

void add_edge(int u, int v) {
  int id_u_v = g[u].size();
  int id_v_u = g[v].size();
  g[u].emplace_back(v, id_v_u); // id of the parent in the
       child's list (g[v][id] -> u)
  g[v].emplace_back(u, id_u_v); // id of the parent in the
       child's list (g[u][id] -> v)
}


int go(int node, int id_parent) {

  state &s = dp[node];

  if (!s.seen) {
    int ans = 1;
    s.partial.assign(g[node].size(), 0); // create the list
         of partial results.
    for (int i = 0; i < int(g[node].size()); i++) {
      int to = g[node][i].to;
      int pid = g[node][i].p_id;
      if (i != id_parent) {
        int tmp = go(to, pid);
        ans += tmp;
        s.partial[i] = tmp;
      }
    }

    s.missing = id_parent;
    s.total = ans;
    s.seen = true;

    return ans;
  } else {

    if (s.missing == id_parent) { // the same id_parent
         than before, so we can not complete the results
         yet
      return s.total;
    }

    if (s.missing != -1) { // only one missing and is
         different of 'id_parent'
      int tmp = go(g[node][s.missing].to,
           g[node][s.missing].p_id);
      s.partial[s.missing] = tmp;
      s.total += tmp;
      s.missing = -1;
    }

    int extra = (id_parent == -1) ? 0 :
         s.partial[id_parent];
    return s.total - extra;
  }
}
```

3 Data structures

3.1 STL order statistics tree

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <bits/stdc++.h>

using namespace __gnu_pbds;
using namespace std;

typedef
tree<
  pair<int,int>,
  null_type,
  less<pair<int,int>>,
  rb_tree_tag,
  tree_order_statistics_node_update>
ordered_set;

main()
 {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    int sz=0;
    cin>>n;
    vector<int> ans(n,0);

    ordered_set t;
    int x,y;
    for(int i=0;i<n;i++)
    {
        cin>>x>>y;
        ans[t.order_of_key({x,++sz})]++;
        t.insert({x,sz});
    }

    for(int i=0;i<n;i++)
        cout<<ans[i]<<'\n';
}

/***
Input
5
1 1
5 1
7 1
3 3
5 5

Output
1
2
1
1
0
***/
```

3.2 heavy light decomposition

```
// Heavy-Light Decomposition
struct TreeDecomposition {
  vector<int> g[MAXN], c[MAXN];
  int s[MAXN]; // subtree size
  int p[MAXN]; // parent id
  int r[MAXN]; // chain root id
  int t[MAXN]; // index used in segtree/bit/...
  int d[MAXN]; // depht
  int ts;

  void dfs(int v, int f) {
    p[v] = f;
    s[v] = 1;
    if (f != -1) d[v] = d[f] + 1;
    else d[v] = 0;

    for (int i = 0; i < g[v].size(); ++i) {
      int w = g[v][i];
      if (w != f) {
        dfs(w, v);
        s[v] += s[w];
      }
    }
  }

  void hld(int v, int f, int k) {
    t[v] = ts++;
    c[k].push_back(v);
    r[v] = k;

    int x = 0, y = -1;
    for (int i = 0; i < g[v].size(); ++i) {
      int w = g[v][i];
      if (w != f) {
        if (s[w] > x) {
          x = s[w];
          y = w;
        }
      }
    }
    if (y != -1) {
      hld(y, v, k);
    }

    for (int i = 0; i < g[v].size(); ++i) {
      int w = g[v][i];
      if (w != f && w != y) {
        hld(w, v, w);
      }
    }
  }

  void init(int n) {
    for (int i = 0; i < n; ++i) {
      g[i].clear();
    }
  }

  void add(int a, int b) {
    g[a].push_back(b);
    g[b].push_back(a);
  }

  void build() {
    ts = 0;
    dfs(0, -1);
    hld(0, 0, 0);
  }
};
```

3.3 persistent seg tree

```
/**
 *  Problems:
 *    http://codeforces.com/contest/813/problem/E
 *
 *  Important:
 *  When using lazy propagation remembert to create new
 *  versions for each push_down operation!!!
 * */

struct node {
  node *l, *r;
  long long acc;
  int flip;

  node (int x) : l(NULL), r(NULL), acc(x), flip(0) {}
  node () : l(NULL), r(NULL), acc(0), flip(0) {}
};

typedef node* pnode;

pnode create(int l, int r) {
  if (l == r) return new node();
  pnode cur = new node();
  int m = (l + r) >> 1;
  cur-> l = create(l, m);
  cur-> r = create(m + 1, r);
  return cur;
}

pnode copy_node(pnode cur) {
  pnode ans = new node();
  *ans = *cur;
  return ans;
}

void push_down(pnode cur, int l, int r) {
  assert(cur);
  if (cur-> flip) {
    int len = r - l + 1;
    cur-> acc = len - cur-> acc;
    if (cur-> l) {
      cur-> l = copy_node(cur-> l);
      cur-> l -> flip ^= 1;
    }
    if (cur-> r) {
      cur-> r = copy_node(cur-> r);
```

```
    cur-> r -> flip ^= 1;
   }
   cur-> flip = 0;
  }
}

int get_val(pnode cur) {
  assert(cur);
  assert((cur-> flip) == 0);
  if (cur) return cur-> acc;
  return 0;
}

pnode update(pnode cur, int l, int r, int at, int what) {
  pnode ans = copy_node(cur);
  if (l == r) {
    assert(l == at);
    ans-> acc = what;
    ans-> flip = 0;
    return ans;
  }
  int m = (l + r) >> 1;
  push_down(ans, l, r);
  if (at <= m) ans-> l = update(ans-> l, l, m, at, what);
  else ans-> r = update(ans-> r, m + 1, r, at, what);

  push_down(ans-> l, l, m);
  push_down(ans-> r, m + 1, r);
  ans-> acc = get_val(ans-> l) + get_val(ans-> r);
  return ans;
}

pnode flip(pnode cur, int l, int r, int a, int b) {
  pnode ans = new node();

  if (cur != NULL) {
    *ans = *cur;
  }
  if (l > b || r < a)
    return ans;

  if (l >= a && r <= b) {
    ans-> flip ^= 1;
    push_down(ans, l, r);
    return ans;
  }

  int m = (l + r) >> 1;
  ans-> l = flip(ans-> l, l, m, a, b);
  ans-> r = flip(ans-> r, m + 1, r, a, b);
  push_down(ans-> l, l, m);
  push_down(ans-> r, m + 1, r);
  ans-> acc = get_val(ans-> l) + get_val(ans-> r);
  return ans;
}

long long get_all(pnode cur, int l, int r) {
  assert(cur);
  push_down(cur, l, r);
  return cur-> acc;
}
```

```
void traverse(pnode cur, int l, int r) {
  if (!cur) return;
  cout << l << " - " << r << " : " << (cur-> acc) << " "
       << (cur-> flip) << endl;
  traverse(cur-> l, l, (l + r) >> 1);
  traverse(cur-> l, 1 + ((l + r) >> 1), r);
}
```

3.4 persistent trie

```
// both tries can be tested with the problem:
     http://codeforces.com/problemset/problem/916/D


// Persistent binary trie (BST for integers)
const int MD = 31;

struct node_bin {
  node_bin *child[2];
  int val;

  node_bin() : val(0) {
    child[0] = child[1] = NULL;
  }
};

typedef node_bin* pnode_bin;

pnode_bin copy_node(pnode_bin cur) {
  pnode_bin ans = new node_bin();
  if (cur) *ans = *cur;
  return ans;
}

pnode_bin modify(pnode_bin cur, int key, int inc, int id =
     MD) {
  pnode_bin ans = copy_node(cur);
  ans->val += inc;
  if (id >= 0) {
    int to = (key >> id) & 1;
    ans->child[to] = modify(ans->child[to], key, inc, id -
         1);
  }
  return ans;
}

int sum_smaller(pnode_bin cur, int key, int id = MD) {
  if (cur == NULL) return 0;
  if (id < 0) return 0; // strictly smaller
  // if (id == - 1) return cur->val; // smaller or equal

  int ans = 0;
  int to = (key >> id) & 1;
  if (to) {
    if (cur->child[0]) ans += cur->child[0]->val;
    ans += sum_smaller(cur->child[1], key, id - 1);
  } else {
    ans = sum_smaller(cur->child[0], key, id - 1);
  }
  return ans;
}


// Persistent trie for strings.
const int MAX_CHILD = 26;
struct node {
  node *child[MAX_CHILD];
  int val;
  node() : val(-1) {
    for (int i = 0; i < MAX_CHILD; i++) {
      child[i] = NULL;
    }
  }
};

typedef node* pnode;

pnode copy_node(pnode cur) {
  pnode ans = new node();
  if (cur) *ans = *cur;
  return ans;
}

pnode set_val(pnode cur, string &key, int val, int id = 0)
     {
  pnode ans = copy_node(cur);
  if (id >= int(key.size())) {
    ans->val = val;
  } else {
    int t = key[id] - 'a';
    ans->child[t] = set_val(ans->child[t], key, val, id +
         1);
  }
  return ans;
}

pnode get(pnode cur, string &key, int id = 0) {
  if (id >= int(key.size()) || !cur)
    return cur;
  int t = key[id] - 'a';
  return get(cur->child[t], key, id + 1);
}
```

3.5 segment tree

```
/**
 * Taken from: http://codeforces.com/blog/entry/18051
 * */

const int MN = 1e5; // limit for array size

struct seg_tree {
  int n; // array size
  int t[2 * MN];

  seg_tree(int _n) : n(_n) {}
```

```
  void clear() {
    memset(t, 0, sizeof t);
  }

  void build() { // build the tree
    for (int i = n - 1; i > 0; --i) t[i] = t[i<<1] +
        t[i<<1|1];
  }

  // Single modification, range query.
  void modify(int p, int value) { // set value at position
      p
    for (t[p += n] = value; p > 1; p >>= 1) t[p>>1] = t[p]
        + t[p^1];
  }

  int query(int l, int r) { // sum on interval [l, r)
    int res = 0;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l&1) res += t[l++];
      if (r&1) res += t[--r];
    }
    return res;
  }
};

// Range modification, single query.

void modify(int l, int r, int value) {
  for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
    if (l&1) t[l++] += value;
    if (r&1) t[--r] += value;
  }
}

int query(int p) {
  int res = 0;
  for (p += n; p > 0; p >>= 1) res += t[p];
  return res;
}

/**
 * If at some point after modifications we need to inspect
      all the
 * elements in the array, we can push all the
      modifications to the
 * leaves using the following code. After that we can just
      traverse
 * elements starting with index n. This way we reduce the
      complexity
 * from O(n log(n)) to O(n) similarly to using build
      instead of n modifications.
 * */

void push() {
  for (int i = 1; i < n; ++i) {
    t[i<<1] += t[i];
    t[i<<1|1] += t[i];
    t[i] = 0;
  }
}
```

```
// Non commutative combiner functions.

void modify(int p, const S& value) {
  for (t[p += n] = value; p >>= 1; ) t[p] =
       combine(t[p<<1], t[p<<1|1]);
}

S query(int l, int r) {
  S resl, resr;
  for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
    if (l&1) resl = combine(resl, t[l++]);
    if (r&1) resr = combine(t[--r], resr);
  }
  return combine(resl, resr);
}


/**
 * segment tree for intervals
 *
 * */

const int MN = 100000 + 100;
struct seg_tree {
  int val[MN * 4 + 4];
  int pending[MN * 4 + 4];

  seg_tree() {
    memset(val, -1, sizeof val);
    memset(pending, -1, sizeof pending);
  }

  void propagate(int node, int b, int e) {
    if (pending[node] != -1) {
      val[node] = pending[node];
      if (b < e) {
        pending[node << 1] = pending[node];
        pending[node << 1 | 1] = pending[node];
      }
      pending[node] = -1;
    }
  }

  void set(int node, int b, int e, int from, int to, int
       v) {
    if (b > to || e < from) return;

    if (b >= from && e <= to) {
      pending[node] = v;
      propagate(node, b, e);
      return;
    }

    int mid = (b + e) >> 1;

    set(node << 1, b, mid, from, to, v);
    set(node << 1 | 1, mid + 1, e, from, to, v);
  }



  int query(int node, int b, int e, int pos) {
    propagate(node, b, e);

    if (b == e && b == pos) {
      return val[node];
    }

    int mid = (b + e) >> 1;
    if (pos <= mid)
      return query(node << 1, b, mid, pos);
    return query(node << 1 | 1, mid + 1, e, pos);
  }

  void set(int from, int to, int v) {
    return set(1, 0, MN - 1, from, to, v);
  }

  int query(int pos) {
    return query(1, 0, MN - 1, pos);
  }
};
```

3.6 sparse table

```
// RMQ.
const int MN = 100000 + 10; // Max number of elements
const int ML = 18; // ceil(log2(MN));

struct st {
  int data[MN];
  int M[MN][ML];
  int n;

  void init(const vector<int> &d) {
    n = d.size();
    for (int i = 0; i < n; ++i)
      data[i] = d[i];

    build();
  }

  void build() {
    for (int i = 0; i < n; ++i)
      M[i][0] = data[i];
    for (int j = 1, p = 2, q = 1; p <= n; ++j, p <<= 1, q
         <<= 1)
      for (int i = 0; i + p - 1 < n; ++i)
        M[i][j] = max(M[i][j - 1], M[i + q][j - 1]);
  }
  int query(int b, int e) {
    int k = log2(e - b + 1);
    return max(M[b][k], M[e + 1 - (1<<k)][k]);
  }
};
```

3.7 trie

```
const int MN = 26; // size of alphabet
const int MS = 100010; // Number of states.

struct trie{
  struct node{
    int c;
    int a[MN];
  };

  node tree[MS];
  int nodes;

  void clear(){
    tree[nodes].c = 0;
    memset(tree[nodes].a, -1, sizeof tree[nodes].a);
    nodes++;
  }

  void init(){
    nodes = 0;
    clear();
  }

  int add(const string &s, bool query = 0){
    int cur_node = 0;
    for(int i = 0; i < s.size(); ++i){
      int id = gid(s[i]);
      if(tree[cur_node].a[id] == -1){
        if(query) return 0;
        tree[cur_node].a[id] = nodes;
        clear();
      }
      cur_node = tree[cur_node].a[id];
    }
    if(!query) tree[cur_node].c++;
    return tree[cur_node].c;
  }

};
```

4 Geometry

4.1 Notes

Area of a polygon is:

$$\frac{1}{2}\sum_{i=1}^{n-1}|p_i \times p_{i+1}| = \frac{1}{2}\sum_{i=1}^{n-1}|x_i y_{i+1} - x_{i+1} y_i|$$

p_i is adjacent to p_{i+1}, and $p_1 = p_n$.

If no absolute, if area is smaller than 0 then points are listed CCW, otherwise CW, if equals 0 then 3 points are in a line.

Point location to a segment. Point is p, segment from a to b $(p-a) \times (p-b) < 0, = 0, > 0$: Left, Touch, Right

4.2 center 2 points + radious

```
vector<point> find_center(point a, point b, long double r)
    {
  point d = (a - b) * 0.5;
  if (d.dot(d) > r * r) {
    return vector<point> ();
  }
  point e = b + d;
  long double fac = sqrt(r * r - d.dot(d));
  vector<point> ans;
  point x = point(-d.y, d.x);
  long double l = sqrt(x.dot(x));
  x = x * (fac / l);
  ans.push_back(e + x);
  x =  point(d.y, -d.x);
  x = x * (fac / l);
  ans.push_back(e + x);
  return ans;
}
```

4.3 closest pair problem

```
struct point {
  double x, y;
  int id;
  point() {}
  point (double a, double b) : x(a), y(b) {}
};

double dist(const point &o, const point &p) {
  double a = p.x - o.x, b = p.y - o.y;
  return sqrt(a * a + b * b);
}

double cp(vector<point> &p, vector<point> &x,
     vector<point> &y) {
  if (p.size() < 4) {
    double best = 1e100;
    for (int i = 0; i < p.size(); ++i)
      for (int j = i + 1; j < p.size(); ++j)
        best = min(best, dist(p[i], p[j]));
    return best;
  }

  int ls = (p.size() + 1) >> 1;
  double l = (p[ls - 1].x + p[ls].x) * 0.5;
  vector<point> xl(ls), xr(p.size() - ls);
  unordered_set<int> left;
  for (int i = 0; i < ls; ++i) {
    xl[i] = x[i];
    left.insert(x[i].id);
  }
  for (int i = ls; i < p.size(); ++i) {
    xr[i - ls] = x[i];
  }

  vector<point> yl, yr;
  vector<point> pl, pr;
  yl.reserve(ls); yr.reserve(p.size() - ls);
  pl.reserve(ls); pr.reserve(p.size() - ls);
  for (int i = 0; i < p.size(); ++i) {
    if (left.count(y[i].id))
      yl.push_back(y[i]);
    else
      yr.push_back(y[i]);

    if (left.count(p[i].id))
      pl.push_back(p[i]);
    else
      pr.push_back(p[i]);
  }

  double dl = cp(pl, xl, yl);
  double dr = cp(pr, xr, yr);
  double d = min(dl, dr);
  vector<point> yp; yp.reserve(p.size());
  for (int i = 0; i < p.size(); ++i) {
    if (fabs(y[i].x - l) < d)
      yp.push_back(y[i]);
  }
  for (int i = 0; i < yp.size(); ++i) {
    for (int j = i + 1; j < yp.size() && j < i + 7; ++j) {
      d = min(d, dist(yp[i], yp[j]));
    }
  }
  return d;
}

double closest_pair(vector<point> &p) {
  vector<point> x(p.begin(), p.end());
  sort(x.begin(), x.end(), [](const point &a, const point
       &b) {
    return a.x < b.x;
  });
  vector<point> y(p.begin(), p.end());
  sort(y.begin(), y.end(), [](const point &a, const point
       &b) {
    return a.y < b.y;
  });
  return cp(p, x, y);
}
```

4.4 convex hull

```
struct Point {
    int x, y;
    Point(int ix = 0, int iy = 0) {
        x = ix;
        y = iy;
    }
};

bool cmp(Point a, Point b) {
    if(a.x == b.x) return a.y < b.y;
    return a.x < b.x;
}
```

```
Point vec(Point p1, Point p2) {
    Point res;
    res.x = p2.x - p1.x;
    res.y = p2.y - p1.y;
    return res;
}

int cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}

int point_location(vector<Point> p) {
    int res = cross(vec(p[0], p[1]), vec(p[0], p[2]));
    if(res == 0) return 0;
    else if(res > 0) return -1; // left
    return 1; // right
}

void solve()
{
    int n; cin >> n;
    vector<Point> a(n);
    for(auto &it : a) cin >> it.x >> it.y;
    sort(a.begin(), a.end(), cmp);

    vector<Point> v1, v2;
    for(int i = 0; i < n; i++) {
        while(v1.size() >= 2 &&
              point_location({v1[(int)v1.size() - 2],
              v1[(int)v1.size() - 1], a[i]}) == -1)
              v1.pop_back();
        v1.push_back(a[i]);
    }
    for(int i = n - 1; i >= 0; i--) {
        while(v2.size() >= 2 &&
              point_location({v2[(int)v2.size() - 2],
              v2[(int)v2.size() - 1], a[i]}) == -1)
              v2.pop_back();
        v2.push_back(a[i]);
    }

    v1.pop_back();
    v2.pop_back();

    cout << v1.size() + v2.size() << '\n';
    for(auto v: {v1, v2}) {
        for(Point p : v) cout << p.x << ' ' << p.y << '\n';
    }
}
```

4.5 squares

```
typedef long double ld;

const ld eps = 1e-12;
int cmp(ld x, ld y = 0, ld tol = eps) {
    return ( x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
}

struct point{
  ld x, y;
  point(ld a, ld b) : x(a), y(b) {}
  point() {}
};

struct square{
  ld x1, x2, y1, y2,
     a, b, c;
  point edges[4];
  square(ld _a, ld _b, ld _c) {
    a = _a, b = _b, c = _c;
    x1 = a - c * 0.5;
    x2 = a + c * 0.5;
    y1 = b - c * 0.5;
    y2 = b + c * 0.5;
    edges[0] = point(x1, y1);
    edges[1] = point(x2, y1);
    edges[2] = point(x2, y2);
    edges[3] = point(x1, y2);
  }
};

ld min_dist(point &a, point &b) {
  ld x = a.x - b.x,
     y = a.y - b.y;
  return sqrt(x * x + y * y);
}

bool point_in_box(square s1, point p) {
  if (cmp(s1.x1, p.x) != 1 && cmp(s1.x2, p.x) != -1 &&
      cmp(s1.y1, p.y) != 1 && cmp(s1.y2, p.y) != -1)
    return true;
  return false;
}

bool inside(square &s1, square &s2) {
  for (int i = 0; i < 4; ++i)
    if (point_in_box(s2, s1.edges[i]))
      return true;

  return false;
}

bool inside_vert(square &s1, square &s2) {
  if ((cmp(s1.y1, s2.y1) != -1 && cmp(s1.y1, s2.y2) != 1)
        ||
      (cmp(s1.y2, s2.y1) != -1 && cmp(s1.y2, s2.y2) != 1))
    return true;
 return false;
}

bool inside_hori(square &s1, square &s2) {
  if ((cmp(s1.x1, s2.x1) != -1 && cmp(s1.x1, s2.x2) != 1)
        ||
      (cmp(s1.x2, s2.x1) != -1 && cmp(s1.x2, s2.x2) != 1))
    return true;
 return false;
}

ld min_dist(square &s1, square &s2) {
  if (inside(s1, s2) || inside(s2, s1))
    return 0;

  ld ans = 1e100;
  for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 4; ++j)
      ans = min(ans, min_dist(s1.edges[i], s2.edges[j]));


  if (inside_hori(s1, s2) || inside_hori(s2, s1)) {
    if (cmp(s1.y1, s2.y2) != -1)
      ans = min(ans, s1.y1 - s2.y2);
    else
    if (cmp(s2.y1, s1.y2) != -1)
      ans = min(ans, s2.y1 - s1.y2);
  }

  if (inside_vert(s1, s2) || inside_vert(s2, s1)) {
    if (cmp(s1.x1, s2.x2) != -1)
      ans = min(ans, s1.x1 - s2.x2);
    else
    if (cmp(s2.x1, s1.x2) != -1)
      ans = min(ans, s2.x1 - s1.x2);
  }

  return ans;
}
```

4.6 triangles

Let a, b, c be length of the three sides of a triangle.

$$p = (a + b + c) * 0.5$$

The inradius is defined by:

$$iR = \sqrt{\frac{(p-a)(p-b)(p-c)}{p}}$$

The radius of its circumcircle is given by the formula:

$$cR = \frac{abc}{\sqrt{(a+b+c)(a+b-c)(a+c-b)(b+c-a)}}$$

5 Graphs

5.1 bridges

```
struct Graph {
  vector<vector<Edge>> g;
  vector<int> vi, low, d, pi, is_b;
  int bridges_computed;

  int ticks, edges;
```

```
Graph(int n, int m) {
  g.assign(n, vector<Edge>());
  is_b.assign(m, 0);
  vi.resize(n);
  low.resize(n);
  d.resize(n);
  pi.resize(n);
  edges = 0;
  bridges_computed = 0;
}

void AddEdge(int u, int v) {
  g[u].push_back(Edge(v, edges));
  g[v].push_back(Edge(u, edges));
  edges++;
}

void Dfs(int u) {
  vi[u] = true;
  d[u] = low[u] = ticks++;
  for (int i = 0; i < (int)g[u].size(); ++i) {
    int v = g[u][i].to;
    if (v == pi[u]) continue;
    if (!vi[v]) {
      pi[v] = u;
      Dfs(v);
      if (d[u] < low[v]) is_b[g[u][i].id] = true;

      low[u] = min(low[u], low[v]);
    } else {
      low[u] = min(low[u], d[v]);
    }
  }
}

// Multiple edges from a to b are not allowed.
// (they could be detected as a bridge).
// If you need to handle this, just count
// how many edges there are from a to b.
void CompBridges() {
  fill(pi.begin(), pi.end(), -1);
  fill(vi.begin(), vi.end(), 0);
  fill(low.begin(), low.end(), 0);
  fill(d.begin(), d.end(), 0);
  ticks = 0;
  for (int i = 0; i < (int)g.size(); ++i)
    if (!vi[i]) Dfs(i);
  bridges_computed = true;
}

map<int, vector<Edge>> BridgesTree() {
  if (!bridges_computed) CompBridges();
  int n = g.size();
  Dsu dsu(g.size());
  for (int i = 0; i < n; i++)
    for (auto e : g[i])
      if (!is_b[e.id]) dsu.Join(i, e.to);

  map<int, vector<Edge>> tree;
  for (int i = 0; i < n; i++)
    for (auto e : g[i])
      if (is_b[e.id])
        tree[dsu.Find(i)].emplace_back(dsu.Find(e.to),
            e.id);

  return tree;
  }
};
```

5.2 dijkstra

```
struct edge {
  int to;
  long long w;
  edge () {}
  edge (int a, long long b) : to(a), w(b) {}
  bool operator < (const edge &o) const {
    return w > o.w;
  }
};

typedef vector<vector<edge>> graph;

const long long inf = 1000000LL * 10000000LL;

pair<vector<int>, vector<long long>> dijkstra(graph &g,
     int start) {
  int n = g.size();
  vector<long long> d(n, inf);
  vector<int> p(n, -1);
  d[start] = 0;
  priority_queue<edge> q;
  q.push(edge(start, 0));

  while (!q.empty()) {
    int node = q.top().to;
    long long dist = q.top().w;
    q.pop();

    if (dist > d[node]) continue;

    for (int i = 0; i < (int)g[node].size(); i++) {
      int to = g[node][i].to;
      long long w_extra = g[node][i].w;

      if (dist + w_extra < d[to]) {
        p[to] = node;
        d[to] = dist + w_extra;
        q.push(edge(to, d[to]));
      }
    }
  }

  return {p, d};
}
```

5.3 directed mst

```
const int inf = 1000000 + 10;

struct edge {
  int u, v, w;
  edge() {}
  edge(int a,int b,int c) : u(a), v(b), w(c) {}
};

/**
 * Computes the minimum spanning tree for a directed graph
 * - edges : Graph description in the form of list of
      edges.
 *    each edge is: From node u to node v with cost w
 * - root : Id of the node to start the DMST.
 * - n    : Number of nodes in the graph.
 * */

int dmst(vector<edge> &edges, int root, int n) {
  int ans = 0;
  int cur_nodes = n;
  while (true) {
    vector<int> lo(cur_nodes, inf), pi(cur_nodes, inf);
    for (int i = 0; i < edges.size(); ++i) {
      int u = edges[i].u, v = edges[i].v, w = edges[i].w;
      if (w < lo[v] and u != v) {
        lo[v] = w;
        pi[v] = u;
      }
    }

    lo[root] = 0;
    for (int i = 0; i < lo.size(); ++i) {
      if (i == root) continue;
      if (lo[i] == inf) return -1;
    }
    int cur_id = 0;
    vector<int> id(cur_nodes, -1), mark(cur_nodes, -1);
    for (int i = 0; i < cur_nodes; ++i) {
      ans += lo[i];
      int u = i;
      while (u != root and id[u] < 0 and mark[u] != i) {
        mark[u] = i;
        u = pi[u];
      }
      if (u != root and id[u] < 0) { // Cycle
         for (int v = pi[u]; v != u; v = pi[v])
           id[v] = cur_id;
         id[u] = cur_id++;
      }
    }

    if (cur_id == 0)
      break;

    for (int i = 0; i < cur_nodes; ++i)
      if (id[i] < 0) id[i] = cur_id++;

    for (int i = 0; i < edges.size(); ++i) {
      int u = edges[i].u, v = edges[i].v, w = edges[i].w;
      edges[i].u = id[u];
      edges[i].v = id[v];
```

```
      if (id[u] != id[v])
        edges[i].w -= lo[v];
    }
    cur_nodes = cur_id;
    root = id[root];
  }

  return ans;
}
```

5.4 eulerian path

```
// Taken from
     https://github.com/lbv/pc-code/blob/master/code/graph.cpp
// Eulerian Trail

struct Euler {
  ELV adj; IV t;
  Euler(ELV Adj) : adj(Adj) {}
  void build(int u) {
    while(! adj[u].empty()) {
      int v = adj[u].front().v;
      adj[u].erase(adj[u].begin());
      build(v);
    }
    t.push_back(u);
  }
};
bool eulerian_trail(IV &trail) {
  Euler e(adj);
  int odd = 0, s = 0;
  /*
     for (int v = 0; v < n; v++) {
     int diff = abs(in[v] - out[v]);
     if (diff > 1) return false;
     if (diff == 1) {
     if (++odd > 2) return false;
     if (out[v] > in[v]) start = v;
     }
     }
     */
  e.build(s);
  reverse(e.t.begin(), e.t.end());
  trail = e.t;
  return true;
}
```

5.5 karp min mean cycle

```
/**
 * Finds the min mean cycle, if you need the max mean cycle
 * just add all the edges with negative cost and print
 * ans * -1
 *
 * test: uva, 11090 - Going in Cycle!!
 * */

const int MN = 1000;
struct edge{
  int v;
  long long w;
  edge(){} edge(int v, int w) : v(v), w(w) {}
};

long long d[MN][MN];
// This is a copy of g because increments the size
// pass as reference if this does not matter.
int karp(vector<vector<edge> > g) {
  int n = g.size();

  g.resize(n + 1); // this is important

  for (int i = 0; i < n; ++i)
    if (!g[i].empty())
      g[n].push_back(edge(i,0));
  ++n;

  for(int i = 0;i<n;++i)
    fill(d[i],d[i]+(n+1),INT_MAX);

  d[n - 1][0] = 0;

  for (int k = 1; k <= n; ++k) for (int u = 0; u < n; ++u)
      {
    if (d[u][k - 1] == INT_MAX) continue;
    for (int i = g[u].size() - 1; i >= 0; --i)
      d[g[u][i].v][k] = min(d[g[u][i].v][k], d[u][k - 1] +
          g[u][i].w);
  }

  bool flag = true;

  for (int i = 0; i < n && flag; ++i)
    if (d[i][n] != INT_MAX)
      flag = false;

  if (flag) {
    return true; // return true if there is no a cycle.
  }

  double ans = 1e15;

  for (int u = 0; u + 1 < n; ++u) {
    if (d[u][n] == INT_MAX) continue;
    double W = -1e15;

    for (int k = 0; k < n; ++k)
      if (d[u][k] != INT_MAX)
        W = max(W, (double)(d[u][n] - d[u][k]) / (n - k));

    ans = min(ans, W);
  }

  // printf("%.2lf\n", ans);
  cout << fixed << setprecision(2) << ans << endl;

  return false;
}
```

5.6 magic max flow

```
const int N = 1000 + 5;
const int MAXA = 1e9;

int a[N][N];
int n, m, p[N];
bool f[N];

int bfs(){
    memset(f, 0, sizeof f);
    memset(p, 0, sizeof p);
    f[1] = true;
    queue <pair<int, int>> q;
    q.push({1, MAXA});
    while (!q.empty()){
        pair<int, int> st = q.front(); q.pop();
        int u = st.first, val = st.second;

        for(int v = 1; v <= n; ++v){
            if (!f[v] && a[u][v]){
                q.push({v, min(val, a[u][v])});
                f[v] = true; p[v] = u;

                if (v == n) return min(val, a[u][v]);
            }
        }
    }
    return 0;
}

int max_flow(){
    int ans = 0, flow = 0;
    while ((flow = bfs())){
        ans += flow;
        int u = n;
        while (u != 1){
            a[p[u]][u] -= flow;
            a[u][p[u]] += flow;

            u = p[u];
        }
    }
    return ans;
}

void solve()
{
    cin >> n >> m;
    for(int i = 1; i <= m; ++i){
        int u, v, w; cin >> u >> v >> w;
        a[u][v] += w;
    }

    cout << max_flow();
}
```

5.7 maximum matching

```
const int N = 100100;

vector<int> G[N];
int seen[N], iteration = 0;
int dist[N], matchL[N], matchR[N];

bool dfs(int u) {
    if(seen[u] == iteration) return 0;
    seen[u] = iteration;

    for(int v : G[u])
        if(matchR[v] == -1) {
            matchL[u] = v; matchR[v] = u;
            return 1;
        }

    for(int v : G[u])
        if(dist[matchR[v]] == dist[u] + 1 &&
             dfs(matchR[v])) {
            matchL[u] = v; matchR[v] = u;
            return 1;
        }

    return 0;
}

int matching(int M, int N, vector<pair<int, int>> E) {
    fill(matchL + 1, matchL + M + 1, -1);
    fill(matchR + 1, matchR + N + 1, -1);

    for(int i = 0; i < (int) E.size(); ++i) {
        int u = E[i].first, v = E[i].second;
        G[u].push_back(v);
    }

    int ans = 0;
    while(true) {
        queue<int> q;
        for(int u = 1; u <= M; ++u) {
            if(matchL[u] == -1) {
                dist[u] = 0; q.push(u);
            } else {
                dist[u] = -1;
            }
        }
        while(q.size()) {
            int u = q.front(); q.pop();
            for(int v : G[u]) {
                if(matchR[v] != -1 && dist[matchR[v]] == -1)
                     {
                    dist[matchR[v]] = dist[u] + 1;
                    q.push(matchR[v]);
                }
            }
        }

        int newMatches = 0;
        ++iteration;
        for(int u = 1; u <= M; ++u) {
            if(matchL[u] == -1) newMatches += dfs(u);
        }
        if(newMatches == 0) break;
        ans += newMatches;
    }

    return ans;
}
```

5.8 minimum path cover in DAG

Given a directed acyclic graph $G = (V, E)$, we are to find the minimum number of vertex-disjoint paths to cover each vertex in V.

We can construct a bipartite graph $G' = (Vout \cup Vin, E')$ from G, where :

$$Vout = \{v \in V : v \ has \ positive \ out - degree\}$$

$$Vin = \{v \in V : v \ has \ positive \ in - degree\}$$

$$E' = \{(u, v) \in Vout \times Vin : (u, v) \in E\}$$

Then it can be shown, via König's theorem, that G' has a matching of size m if and only if there exists $n - m$ vertex-disjoint paths that cover each vertex in G, where n is the number of vertices in G and m is the maximum cardinality bipartite mathching in G'.

Therefore, the problem can be solved by finding the maximum cardinality matching in G' instead.

NOTE: If the paths are note necesarily disjoints, find the transitive closure and solve the problem for disjoint paths.

5.9 query with lca

```
struct lowest_ca {
  int T[MN], L[MN], W[MN];
  int P[MN][ML], MI[MN][ML], MA[MN][ML];

  void dfs(vector<vector<edge> > &g, int root, int pi =
       -1) {
    if (pi == -1) {
      L[root] = W[root] = 0;
      T[root] = -1;
    }
    for (int i = 0; i < (int)g[root].size(); ++i) {
      int to = g[root][i].v;
      if (to != pi) {
        T[to] = root;
        W[to] = g[root][i].w;
        L[to] = L[root] + 1;
        dfs(g, to, root);
      }
    }
  }

  void init(vector<vector<edge> > &g, int root) {
    // g is undirected
    dfs(g, root);
    int N = g.size(), i, j;

    for (i = 0; i < N; i++) {
      for (j = 0; 1 << j < N; j++) {
        P[i][j] = -1;
        MI[i][j] = inf;
      }
    }

    for (i = 0; i < N; i++) {
      P[i][0] = T[i];
      MI[i][0] = W[i];
    }

    for (j = 1; 1 << j < N; j++)
      for (i = 0; i < N; i++)
        if (P[i][j - 1] != -1) {
          P[i][j] = P[P[i][j - 1]][j - 1];
          MI[i][j] = min(MI[i][j-1], MI[ P[i][j - 1] ][j -
               1]);
        }
  }

  int query(int p, int q) {
    int tmp, log, i;

    int mmin = inf;
    if (L[p] < L[q])
      tmp = p, p = q, q = tmp;

    for (log = 1; 1 << log <= L[p]; log++);
    log--;

    for (i = log; i >= 0; i--)
      if (L[p] - (1 << i) >= L[q]) {
        mmin = min(mmin, MI[p][i]);
        p = P[p][i];
      }

    if (p == q)
      // return p;
      return mmin;

    for (i = log; i >= 0; i--)
      if (P[p][i] != -1 && P[p][i] != P[q][i]) {
        mmin = min(mmin, min(MI[p][i], MI[q][i]));
        p = P[p][i], q = P[q][i];
      }

    // return T[p];
    return min(mmin, min(MI[p][0], MI[q][0]));
  }

  int get_child(int p, int q) { // p is ancestor of q
    if (p == q) return -1;

    int i, log;
    for (log = 1; 1 << log <= L[q]; log++) {}
```

```
    log--;

    for (i = log; i >= 0; i--)
      if (L[q] - (1 << i) > L[p]) {
        q = P[q][i];
      }

    assert(P[q][0] == p);
    return q;
  }

  int is_ancestor(int p, int q) {
    if (L[p] >= L[q])
      return false;

    int dist = L[q] - L[p];

    int cur = q;
    int step = 0;
    while (dist) {
      if (dist & 1)
        cur = P[cur][step];
      step++;
      dist >>= 1;
    }

    return cur == p;
  }
};
```

5.10 tarjan scc

```
const int MN = 20002;

struct tarjan_scc {
  int scc[MN], low[MN], d[MN], stacked[MN];
  int ticks, current_scc;
  deque<int> s; // used as stack.

  tarjan_scc() {}

  void init () {
    memset(scc, -1, sizeof scc);
    memset(d, -1, sizeof d);
    memset(stacked, 0, sizeof stacked);
    s.clear();
    ticks = current_scc = 0;
  }

  void compute(vector<vector<int> > &g, int u) {
    d[u] = low[u] = ticks++;
    s.push_back(u);
    stacked[u] = true;
    for (int i = 0; i < g[u].size(); ++i) {
      int v = g[u][i];
      if (d[v] == -1)
        compute(g, v);
      if (stacked[v]) {
        low[u] = min(low[u], low[v]);
      }
    }

    if (d[u] == low[u]) { // root
      int v;
      do {
        v = s.back();s.pop_back();
        stacked[v] = false;
        scc[v] = current_scc;
      } while (u != v);
      current_scc++;
    }
  }
};
```

5.11 two sat (with kosaraju)

```
/**
 *  Given a set of clauses (a1 v a2)^(a2 v a3)....
 *  this algorithm find a solution to it set of clauses.
 *  test:
      http://lightoj.com/volume_showproblem.php?problem=1251
 **/

#include<bits/stdc++.h>
using namespace std;
#define MAX 100000
#define endl '\n'

vector<int> G[MAX];
vector<int> GT[MAX];
vector<int> Ftime;
vector<vector<int> > SCC;
bool visited[MAX];
int n;


void dfs1(int n){
  visited[n] = 1;

  for (int i = 0; i < G[n].size(); ++i) {
    int curr = G[n][i];
    if (visited[curr]) continue;
    dfs1(curr);
  }

  Ftime.push_back(n);
}

void dfs2(int n, vector<int> &scc) {
  visited[n] = 1;
  scc.push_back(n);

  for (int i = 0;i < GT[n].size(); ++i) {
    int curr = GT[n][i];
    if (visited[curr]) continue;
    dfs2(curr, scc);
  }
}

void kosaraju() {
  memset(visited, 0, sizeof visited);

  for (int i = 0; i < 2 * n ; ++i) {
    if (!visited[i]) dfs1(i);
  }

  memset(visited, 0, sizeof visited);
  for (int i = Ftime.size() - 1; i >= 0; i--) {
    if (visited[Ftime[i]]) continue;
    vector<int> _scc;
    dfs2(Ftime[i],_scc);
    SCC.push_back(_scc);
  }
}

/**
 * After having the SCC, we must traverse each scc, if in
      one SCC are -b y b, there is not a solution.
 * Otherwise we build a solution, making the first "node"
      that we find truth and its complement false.
 **/


bool two_sat(vector<int> &val) {
  kosaraju();
  for (int i = 0; i < SCC.size(); ++i) {
    vector<bool> tmpvisited(2 * n, false);
    for (int j = 0; j < SCC[i].size(); ++j) {
      if (tmpvisited[SCC[i][j] ^ 1]) return 0;
      if (val[SCC[i][j]] != -1) continue;
      else {
        val[SCC[i][j]] = 0;
        val[SCC[i][j] ^ 1] = 1;
      }
      tmpvisited[SCC[i][j]] = 1;
    }
  }
  return 1;
}

// Example of use

int main() {

  int m, u, v, nc = 0, t; cin >> t;
  // n = "nodes" number, m = clauses number

  while (t--) {
    cin >> m >> n;
    Ftime.clear();
    SCC.clear();
    for (int i = 0; i < 2 * n; ++i) {
      G[i].clear();
      GT[i].clear();
    }

    // (a1 v a2) = (a1 -> a2) = (a2 -> a1)
    for (int i = 0; i < m ; ++i) {
      cin >> u >> v;
```

```
    int t1 = abs(u) - 1;
    int t2 = abs(v) - 1;
    int p = t1 * 2 + ((u < 0)? 1 : 0);
    int q = t2 * 2 + ((v < 0)? 1 : 0);
    G[p ^ 1].push_back(q);
    G[q ^ 1].push_back(p);
    GT[p].push_back(q ^ 1);
    GT[q].push_back(p ^ 1);
  }

  vector<int> val(2 * n, -1);
  cout << "Case " << ++nc <<": ";
  if (two_sat(val)) {
    cout << "Yes" << endl;
    vector<int> sol;
    for (int i = 0; i < 2 * n; ++i)
      if (i % 2 == 0 and val[i] == 1)
        sol.push_back(i / 2 + 1);
    cout << sol.size() ;

    for (int i = 0; i < sol.size(); ++i) {
      cout << " " << sol[i];
    }
    cout << endl;
  } else {
    cout << "No" << endl;
  }
 }
 return 0;
}
```

6 Math

6.1 Lucas theorem

For non-negative integers m and n and a prime p, the following congruence relation holds: :

$$\binom{m}{n} \equiv \prod_{i=0}^{k} \binom{m_i}{n_i} \pmod{p},$$

where :

$$m = m_k p^k + m_{k-1} p^{k-1} + \cdots + m_1 p + m_0,$$

and :

$$n = n_k p^k + n_{k-1} p^{k-1} + \cdots + n_1 p + n_0$$

are the base p expansions of m and n respectively. This uses the convention that $\binom{m}{n} = 0$ if $m \leq n$.

6.2 cumulative sum of divisors

```
/**
SOD(n) = sum of actual divisors, CSOD(n) = sod(1) + sod(2)
     + ... + sod(n)
*/

long long csod(long long n) {
  long long ans = 0;
  for (long long i = 2; i * i <= n; ++i) {
    long long j = n / i;
    ans += (i + j) * (j - i + 1) / 2;
    ans += i * (j - i);
  }
  return ans;
}
```

6.3 fibonacci properties

Let A, B and n be integer numbers.

$$k = A - B \tag{1}$$

$$F_A F_B = F_{k+1} F_A^2 + F_k F_A F_{A-1} \tag{2}$$

$$\sum_{i=0}^{n} F_i^2 = F_{n+1} F_n \tag{3}$$

$ev(n) =$ returns 1 if n is even.

$$\sum_{i=0}^{n} F_i F_{i+1} = F_{n+1}^2 - ev(n) \tag{4}$$

$$\sum_{i=0}^{n} F_i F_{i-1} = \sum_{i=0}^{n-1} F_i F_{i+1} \tag{5}$$

6.4 sigma function

the sigma function is defined as:

$$\sigma_x(n) = \sum_{d|n} d^x$$

when $x = 0$ is called the divisor function, that counts the number of positive divisors of n.

Now, we are interested in find

$$\sum_{d|n} \sigma_0(d)$$

if n is written as prime factorization:

$$n = \prod_{i=1}^{k} P_i^{e_k}$$

we can demonstrate that:

$$\sum_{d|n} \sigma_0(d) = \prod_{i=1}^{k} g(e_k + 1)$$

where $g(x)$ is the sum of the first x positive numbers:

$$g(x) = (x * (x + 1))/2$$

6.5 system of equations

```
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be
     infinity or a big number

int gauss (vector < vector<double> > a, vector<double> &
     ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

7 Matrix

7.1 matrix

```
const int MN = 111;
const int mod = 10000;

struct matrix {
  int r, c;
  int m[MN][MN];

  matrix (int _r, int _c) : r (_r), c (_c) {
    memset(m, 0, sizeof m);
  }

  void print() {
    for (int i = 0; i < r; ++i) {
      for (int j = 0; j < c; ++j)
        cout << m[i][j] << " ";
      cout << endl;
    }
  }

  int x[MN][MN];
  matrix & operator *= (const matrix &o) {
    memset(x, 0, sizeof x);
    for (int i = 0; i < r; ++i)
      for (int k = 0; k < c; ++k)
        if (m[i][k] != 0)
          for (int j = 0; j < c; ++j) {
            x[i][j] = (x[i][j] + ((m[i][k] * o.m[k][j]) %
                 mod) ) % mod;
          }
    memcpy(m, x, sizeof(m));
    return *this;
  }
};

void matrix_pow(matrix b, long long e, matrix &res) {
  memset(res.m, 0, sizeof res.m);
  for (int i = 0; i < b.r; ++i)
    res.m[i][i] = 1;

  if (e == 0) return;
  while (true) {
    if (e & 1) res *= b;
    if ((e >>= 1) == 0) break;
    b *= b;
  }
}
```

8 Misc

8.1 dates and bit

```
//
// Time - Leap years
//

// A[i] has the accumulated number of days from months
     previous to i
const int A[13] = { 0, 0, 31, 59, 90, 120, 151, 181, 212,
     243, 273, 304, 334 };
// same as A, but for a leap year
const int B[13] = { 0, 0, 31, 60, 91, 121, 152, 182, 213,
     244, 274, 305, 335 };
// returns number of leap years up to, and including, y
int leap_years(int y) { return y / 4 - y / 100 + y / 400; }
bool is_leap(int y) { return y % 400 == 0 || (y % 4 == 0
     && y % 100 != 0); }
// number of days in blocks of years
const int p400 = 400*365 + leap_years(400);
const int p100 = 100*365 + leap_years(100);
const int p4  = 4*365 + 1;
const int p1  = 365;
int date_to_days(int d, int m, int y)
{
  return (y - 1) * 365 + leap_years(y - 1) + (is_leap(y) ?
       B[m] : A[m]) + d;
}
void days_to_date(int days, int &d, int &m, int &y)
{
  bool top100; // are we in the top 100 years of a 400
       block?
  bool top4;   // are we in the top 4 years of a 100 block?
  bool top1;   // are we in the top year of a 4 block?

  y = 1;
  top100 = top4 = top1 = false;

  y += ((days-1) / p400) * 400;
  d = (days-1) % p400 + 1;

  if (d > p100*3) top100 = true, d -= 3*p100, y += 300;
  else y += ((d-1) / p100) * 100, d = (d-1) % p100 + 1;

  if (d > p4*24) top4 = true, d -= 24*p4, y += 24*4;
  else y += ((d-1) / p4) * 4, d = (d-1) % p4 + 1;

  if (d > p1*3) top1 = true, d -= p1*3, y += 3;
  else y += (d-1) / p1, d = (d-1) % p1 + 1;

  const int *ac = top1 && (!top4 || top100) ? B : A;
  for (m = 1; m < 12; ++m) if (d <= ac[m + 1]) break;
  d -= ac[m];
}

// Bit manipulation
/*
 * n | (1 << x) sets the x-th bit
 * n ^ (1 << x) flips the x-th bit
 * n & ~(1 << x) clears the x-th bit
*/
bool is_set(unsigned int number, int x) {
    return (number >> x) & 1;
}

bool isDivisibleByPowerOf2(int n, int k) {
    int powerOf2 = 1 << k;
    return (n & (powerOf2 - 1)) == 0;
}

bool isPowerOfTwo(unsigned int n) {
    return n && !(n & (n - 1));
}
```

9 Number theory

9.1 crt

```
/**
 * Chinese remainder theorem.
 * Find z such that z % x[i] = a[i] for all i.
 * */
long long crt(vector<long long> &a, vector<long long> &x) {
  long long z = 0;
  long long n = 1;
  for (int i = 0; i < x.size(); ++i)
    n *= x[i];

  for (int i = 0; i < a.size(); ++i) {
    long long tmp = (a[i] * (n / x[i])) % n;
    tmp = (tmp * mod_inv(n / x[i], x[i])) % n;
    z = (z + tmp) % n;
  }

  return (z + n) % n;
}
```

9.2 diophantine equations

```
long long gcd(long long a, long long b, long long &x, long
     long &y) {
  if (a == 0) {
    x = 0;
    y = 1;
    return b;
  }
  long long x1, y1;
  long long d = gcd(b % a, a, x1, y1);
  x = y1 - (b / a) * x1;
  y = x1;
  return d;
}

bool find_any_solution(long long a, long long b, long long
     c, long long &x0,
    long long &y0, long long &g) {
  g = gcd(abs(a), abs(b), x0, y0);
  if (c % g) {
    return false;
  }

  x0 *= c / g;
```

```
  y0 *= c / g;
  if (a < 0) x0 = -x0;
  if (b < 0) y0 = -y0;
  return true;
}

void shift_solution(long long &x, long long &y, long long
     a, long long b,
    long long cnt) {
  x += cnt * b;
  y -= cnt * a;
}

long long find_all_solutions(long long a, long long b,
     long long c,
    long long minx, long long maxx, long long miny,
    long long maxy) {
  long long x, y, g;
  if (!find_any_solution(a, b, c, x, y, g)) return 0;
  a /= g;
  b /= g;

  long long sign_a = a > 0 ? +1 : -1;
  long long sign_b = b > 0 ? +1 : -1;

  shift_solution(x, y, a, b, (minx - x) / b);
  if (x < minx) shift_solution(x, y, a, b, sign_b);
  if (x > maxx) return 0;
  long long lx1 = x;

  shift_solution(x, y, a, b, (maxx - x) / b);
  if (x > maxx) shift_solution(x, y, a, b, -sign_b);
  long long rx1 = x;

  shift_solution(x, y, a, b, -(miny - y) / a);
  if (y < miny) shift_solution(x, y, a, b, -sign_a);
  if (y > maxy) return 0;
  long long lx2 = x;

  shift_solution(x, y, a, b, -(maxy - y) / a);
  if (y > maxy) shift_solution(x, y, a, b, sign_a);
  long long rx2 = x;

  if (lx2 > rx2) swap(lx2, rx2);
  long long lx = max(lx1, lx2);
  long long rx = min(rx1, rx2);

  if (lx > rx) return 0;
  return (rx - lx) / abs(b) + 1;
}
```

9.3 discrete logarithm

```
// Computes x which a ^ x = b mod n.

long long d_log(long long a, long long b, long long n) {
  long long m = ceil(sqrt(n));
  long long aj = 1;
  map<long long, long long> M;
  for (int i = 0; i < m; ++i) {
    if (!M.count(aj))
      M[aj] = i;
    aj = (aj * a) % n;
  }

  long long coef = mod_pow(a, n - 2, n);
  coef = mod_pow(coef, m, n);
  // coef = a ^ (-m)
  long long gamma = b;
  for (int i = 0; i < m; ++i) {
    if (M.count(gamma)) {
      return i * m + M[gamma];
    } else {
      gamma = (gamma * coef) % n;
    }
  }
  return -1;
}
```

9.4 ext euclidean

```
void ext_euclid(long long a, long long b, long long &x,
     long long &y, long long &g) {
  x = 0, y = 1, g = b;
  long long m, n, q, r;
  for (long long u = 1, v = 0; a != 0; g = a, a = r) {
    q = g / a, r = g % a;
    m = x - u * q, n = y - v * q;
    x = u, y = v, u = m, v = n;
  }
}
```

9.5 highest exponent factorial

```
/*
 * Largest k so that n! is divisible by p^k
 * If k is prime then use algorithim below
 * If not, we have k = k = k_1^{p_1} \cdot \ldots \cdot
     k_m^{p_m}
 * Then the answer will be min(highest_exponent(k_i, n) /
     p_i) with i = 1...m
*/
int highest_exponent(int p, const int &n){
  int ans = 0;
  int t = p;
  while(t <= n){
    ans += n/t;
    t*=p;
  }
  return ans;
}
```

9.6 miller rabin

```
const int rounds = 20;

// checks whether a is a witness that n is not prime, 1 <
     a < n
bool witness(long long a, long long n) {
  // check as in Miller Rabin Primality Test described
  long long u = n - 1;
  int t = 0;
  while (u % 2 == 0) {
    t++;
    u >>= 1;
  }
  long long next = mod_pow(a, u, n);
  if (next == 1) return false;
  long long last;
  for (int i = 0; i < t; ++i) {
    last = next;
    next = mod_mul(last, last, n);
    if (next == 1) {
      return last != n - 1;
    }
  }
  return next != 1;
}


// Checks if a number is prime with prob 1 - 1 / (2 ^ it)
//  D(miller_rabin(9999999999999997LL) == 1);
//  D(miller_rabin(9999999999971LL) == 1);
//  D(miller_rabin(7907) == 1);
bool miller_rabin(long long n, int it = rounds) {
  if (n <= 1) return false;
  if (n == 2) return true;
  if (n % 2 == 0) return false;
  for (int i = 0; i < it; ++i) {
    long long a = rand() % (n - 1) + 1;
    if (witness(a, n)) {
      return false;
    }
  }
  return true;
}
```

9.7 mod integer

```
template<class T, T mod>
struct mint_t {
  T val;
  mint_t() : val(0) {}
  mint_t(T v) : val(v % mod) {}

  mint_t operator + (const mint_t& o) const {
    return (val + o.val) % mod;
  }
  mint_t operator - (const mint_t& o) const {
    return (val - o.val) % mod;
```

```
  }
  mint_t operator * (const mint_t& o) const {
    return (val * o.val) % mod;
  }
};

typedef mint_t<long long, 998244353> mint;
```

9.8 mod inv

```
long long mod_inv(long long n, long long m) {
  long long x, y, gcd;
  ext_euclid(n, m, x, y, gcd);
  if (gcd != 1)
    return 0;
  return (x + m) % m;
}
```

9.9 mod mul

```
// Computes (a * b) % mod
long long mod_mul(long long a, long long b, long long mod)
    {
  long long x = 0, y = a % mod;
  while (b > 0) {
    if (b & 1)
      x = (x + y) % mod;
    y = (y * 2) % mod;
    b /= 2;
  }
  return x % mod;
}
```

9.10 primes

```
namespace primes {
  const int MP = 100001;
  bool sieve[MP];
  long long primes[MP];
  int num_p;
  void fill_sieve() {
    num_p = 0;
    sieve[0] = sieve[1] = true;
    for (long long i = 2; i < MP; ++i) {
      if (!sieve[i]) {
        primes[num_p++] = i;
        for (long long j = i * i; j < MP; j += i)
          sieve[j] = true;
      }
    }
  }
```

```
  // Finds prime numbers between a and b, using basic
       primes up to sqrt(b)
  // a must be greater than 1.
  vector<long long> seg_sieve(long long a, long long b) {
    long long ant = a;
    a = max(a, 3LL);
    vector<bool> pmap(b - a + 1);
    long long sqrt_b = sqrt(b);
    for (int i = 0; i < num_p; ++i) {
      long long p = primes[i];
      if (p > sqrt_b) break;
      long long j = (a + p - 1) / p;
      for (long long v = (j == 1) ? p + p : j * p; v <= b;
           v += p) {
        pmap[v - a] = true;
      }
    }
    vector<long long> ans;
    if (ant == 2) ans.push_back(2);
    int start = a % 2 ? 0 : 1;
    for (int i = start, I = b - a + 1; i < I; i += 2)
      if (pmap[i] == false)
        ans.push_back(a + i);
    return ans;
  }

  vector<pair<int, int>> factor(int n) {
    vector<pair<int, int>> ans;
    if (n == 0) return ans;
    for (int i = 0; primes[i] * primes[i] <= n; ++i) {
      if ((n % primes[i]) == 0) {
        int expo = 0;
        while ((n % primes[i]) == 0) {
          expo++;
          n /= primes[i];
        }
        ans.emplace_back(primes[i], expo);
      }
    }

    if (n > 1) {
      ans.emplace_back(n, 1);
    }
    return ans;
  }
}
```

9.11 totient sieve

```
  for (int i = 1; i < MN; i++)
    phi[i] = i;

  for (int i = 1; i < MN; i++)
    if (!sieve[i]) // is prime
      for (int j = i; j < MN; j += i)
        phi[j] -= phi[j] / i;
```

9.12 totient

```
long long totient(long long n) {
  if (n == 1) return 0;
  long long ans = n;
  for (int i = 0; primes[i] * primes[i] <= n; ++i) {
    if ((n % primes[i]) == 0) {
      while ((n % primes[i]) == 0) n /= primes[i];
      ans -= ans / primes[i];
    }
  }
  if (n > 1) {
    ans -= ans / n;
  }
  return ans;
}
```

10 Strings

10.1 Incremental Aho Corasick

```
class IncrementalAhoCorasic {
  static const int Alphabets = 26;
  static const int AlphabetBase = 'a';
  struct Node {
    Node *fail;
    Node *next[Alphabets];
    int sum;
    Node() : fail(NULL), next{}, sum(0) { }
  };

  struct String {
    string str;
    int sign;
  };

public:
  //totalLen = sum of (len + 1)
  void init(int totalLen) {
    nodes.resize(totalLen);
    nNodes = 0;
    strings.clear();
    roots.clear();
    sizes.clear();
    que.resize(totalLen);
  }

  void insert(const string &str, int sign) {
    strings.push_back(String{ str, sign });
    roots.push_back(nodes.data() + nNodes);
    sizes.push_back(1);
    nNodes += (int)str.size() + 1;
    auto check = [&]() { return sizes.size() > 1 &&
         sizes.end()[-1] == sizes.end()[-2]; };
    if(!check())
      makePMA(strings.end() - 1, strings.end(),
           roots.back(), que);
```

```
    while(check()) {
      int m = sizes.back();
      roots.pop_back();
      sizes.pop_back();
      sizes.back() += m;
      if(!check())
        makePMA(strings.end() - m * 2, strings.end(),
            roots.back(), que);
    }
  }

  int match(const string &str) const {
    int res = 0;
    for(const Node *t : roots)
      res += matchPMA(t, str);
    return res;
  }

private:
  static void makePMA(vector<String>::const_iterator
      begin, vector<String>::const_iterator end, Node
      *nodes, vector<Node*> &que) {
    int nNodes = 0;
    Node *root = new(&nodes[nNodes ++]) Node();
    for(auto it = begin; it != end; ++ it) {
      Node *t = root;
      for(char c : it->str) {
        Node *&n = t->next[c - AlphabetBase];
        if(n == nullptr)
          n = new(&nodes[nNodes ++]) Node();
        t = n;
      }
      t->sum += it->sign;
    }
    int qt = 0;
    for(Node *&n : root->next) {
      if(n != nullptr) {
        n->fail = root;
        que[qt ++] = n;
      } else {
        n = root;
      }
    }
    for(int qh = 0; qh != qt; ++ qh) {
      Node *t = que[qh];
      int a = 0;
      for(Node *n : t->next) {
        if(n != nullptr) {
          que[qt ++] = n;
          Node *r = t->fail;
          while(r->next[a] == nullptr)
            r = r->fail;
          n->fail = r->next[a];
          n->sum += r->next[a]->sum;
        }
        ++ a;
      }
    }
  }

  static int matchPMA(const Node *t, const string &str) {
    int res = 0;
    for(char c : str) {
      int a = c - AlphabetBase;
      while(t->next[a] == nullptr)
        t = t->fail;
      t = t->next[a];
      res += t->sum;
    }
    return res;
  }


  vector<Node> nodes;
  int nNodes;
  vector<String> strings;
  vector<Node*> roots;
  vector<int> sizes;
  vector<Node*> que;
};

int main() {
  int m;
  while(~scanf("%d", &m)) {
    IncrementalAhoCorasic iac;
    iac.init(600000);
    rep(i, m) {
      int ty;
      char s[300001];
      scanf("%d%s", &ty, s);
      if(ty == 1) {
        iac.insert(s, +1);
      } else if(ty == 2) {
        iac.insert(s, -1);
      } else if(ty == 3) {
        int ans = iac.match(s);
        printf("%d\n", ans);
        fflush(stdout);
      } else {
        abort();
      }
    }
  }
  return 0;
}
```

10.2 kmp

```
// pi[i] = max(k = 0...i, s[0...k - 1] = s[i - (k -
    1)...i])
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
```

10.3 manacher

```
/*
 * Longest palindrome substring in O(n)
*/
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 1, r = 1;
    for(int i = 1; i <= n; i++) {
        p[i] = max(0, min(r - i, p[l + (r - i)]));
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}
vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}
```

10.4 minimal string rotation

```
// Lexicographically minimal string rotation
int lmsr() {
  string s;
  cin >> s;
  int n = s.size();
  s += s;
  vector<int> f(s.size(), -1);
  int k = 0;
  for (int j = 1; j < 2 * n; ++j) {
    int i = f[j - k - 1];
    while (i != -1 && s[j] != s[k + i + 1]) {
      if (s[j] < s[k + i + 1])
        k = j - i - 1;
      i = f[i];
    }
    if (i == -1 && s[j] != s[k + i + 1]) {
      if (s[j] < s[k + i + 1]) {
        k = j;
      }
      f[j - k] = -1;
    } else {
```

```
      f[j - k] = i + 1;
    }
  }
  return k;
}
```

10.5 suffix array

```
/**
 * O (n log^2 (n))
 * See
      http://web.stanford.edu/class/cs97si/suffix-array.pdf
      for reference
 * */

struct entry{
  int a, b, p;
  entry(){}
  entry(int x, int y, int z): a(x), b(y), p(z){}
  bool operator < (const entry &o) const {
    return (a == o.a) ? (b == o.b) ? ( p < o.p) : (b < o.b)
         : (a < o.a);
  }
};

struct SuffixArray{
  const int N;
  string s;
  vector<vector<int> > P;
  vector<entry> M;

  SuffixArray(const string &s) : N(s.length()) , s(s),
       P(1, vector<int> (N, 0)), M(N) {
    for (int i = 0; i < N; ++i)
      P[0][i] = (int) s[i];

    for (int skip = 1, level = 1; skip < N; skip *= 2,
         level++) {
      P.push_back(vector<int>(N, 0));
      for (int i = 0 ; i < N; ++i) {
        int next = ((i + skip) < N) ? P[level - 1][i +
             skip] : -10000;
        M[i] = entry(P[level - 1][i], next, i);
      }
      sort(M.begin(), M.end());
      for (int i = 0; i < N; ++i)
        P[level][M[i].p] = (i > 0 and M[i].a == M[i - 1].a
             and M[i].b == M[i - 1].b) ? P[level][M[i -
             1].p] : i;
    }
  }

  vector<int> getSuffixArray(){
    vector<int> &rank = P.back();
    vector<pair<int, int> > inv(rank.size());
    for (int i = 0; i < rank.size(); ++i)
      inv[i] = make_pair(rank[i], i);
    sort(inv.begin(), inv.end());
    vector<int> sa(rank.size());
    for (int i = 0; i < rank.size(); ++i)
      sa[i] = inv[i].second;
    return sa;
  }

  // returns the length of the longest common prefix of
       s[i...L-1] and s[j...L-1]
  int lcp(int i, int j) {
    int len = 0;
    if (i == j) return N - i;
    for (int k = P.size() - 1; k >= 0 && i < N && j < N;
         --k) {
      if (P[k][i] == P[k][j]) {
        i += 1 << k;
        j += 1 << k;
        len += 1 << k;
      }
    }
    return len;
  }
};
```

10.6 suffix automaton

```
/*
 * Suffix automaton:
 * This implementation was extended to maintain (online)
      the
 * number of different substrings. This is equivalent to
      compute
 * the number of paths from the initial state to all the
      other
 * states.
 *
 * The overall complexity is O(n)
 * can be tested here:
      https://www.urionlinejudge.com.br/judge/en/problems/view/1530
 * */

struct state {
  int len, link;
  long long num_paths;
  map<int, int> next;
};

const int MN = 200011;
state sa[MN << 1];
int sz, last;
long long tot_paths;

void sa_init() {
  sz = 1;
  last = 0;
  sa[0].len = 0;
  sa[0].link = -1;
  sa[0].next.clear();
  sa[0].num_paths = 1;
  tot_paths = 0;
}

void sa_extend(int c) {
  int cur = sz++;
  sa[cur].len = sa[last].len + 1;
  sa[cur].next.clear();
  sa[cur].num_paths = 0;
  int p;
  for (p = last; p != -1 && !sa[p].next.count(c); p =
       sa[p].link) {
    sa[p].next[c] = cur;
    sa[cur].num_paths += sa[p].num_paths;
    tot_paths += sa[p].num_paths;
  }

  if (p == -1) {
    sa[cur].link = 0;
  } else {
    int q = sa[p].next[c];
    if (sa[p].len + 1 == sa[q].len) {
      sa[cur].link = q;
    } else {
      int clone = sz++;
      sa[clone].len = sa[p].len + 1;
      sa[clone].next = sa[q].next;
      sa[clone].num_paths = 0;
      sa[clone].link = sa[q].link;
      for (; p!= -1 && sa[p].next[c] == q; p = sa[p].link) {
        sa[p].next[c] = clone;
        sa[q].num_paths -= sa[p].num_paths;
        sa[clone].num_paths += sa[p].num_paths;
      }
      sa[q].link = sa[cur].link = clone;
    }
  }
  last = cur;
}
```

10.7 z algorithm

```
vector<int> compute_z(const string &s){
  int n = s.size();
  vector<int> z(n,0);
  int l,r;
  r = l = 0;
  for(int i = 1; i < n; ++i){
    if(i > r) {
      l = r = i;
      while(r < n and s[r - l] == s[r])r++;
      z[i] = r - l;r--;
    }else{
      int k = i-l;
      if(z[k] < r - i +1) z[i] = z[k];
      else {
        l = i;
        while(r < n and s[r - l] == s[r])r++;
        z[i] = r - l;r--;
      }
    }
  }
```

```
  return z;
}
/*
 * alfalfa
 * 0 0 0 4 0 0 1
 * z[0] = 0, z[i] = max(k : s[0..k] = s[i...i + k - 1])
*/
```

11 X - Extra Math

11.1 equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

11.2 Trigonometry and Triangles

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ cos(v + w) &= \cos v \cos w - \sin v \sin w \\ \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two): $s_a = \sqrt{bc\left[1 - \left(\frac{a}{b + c}\right)^2\right]}$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

11.3 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

11.4 Spherical coordinates

$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \text{acos}(z/\sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

11.5 Sums and (Geometric) series

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n + 1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n + 1)(n + 1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n + 1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

$$r \neq 1$$

$$a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} = \sum_{k=0}^{n-1} ar^k = a\left(\frac{1 - r^n}{1 - r}\right)$$

11.6 Combinatorics

Binomial coefficients

$$\binom{n}{k} = \frac{n!}{k!(n - k)!}, \quad \binom{n}{0} = \binom{n}{n} = 1$$

$$\binom{n}{k} = \binom{n - 1}{k - 1} + \binom{n - 1}{k} \quad \text{(Pascal's rule)}$$

$$\binom{n}{k} = \binom{n}{n - k}$$

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n, \quad \sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0$$

$$\sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1}, \quad \sum_{k=0}^{n} k^2\binom{n}{k} = n(n + 1)2^{n-2}$$

$$\binom{r}{k} = \frac{r(r - 1)\ldots(r - k + 1)}{k!}$$

Binomial theorem

$$(x + y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k$$

$$(1 + x)^r = \sum_{k=0}^{\infty} \binom{r}{k} x^k, \quad |x| < 1$$

Counting formulas

$$P(n,k) = \frac{n!}{(n-k)!} \quad \text{(permutations of } k \text{ from } n\text{)}$$

$$C(n,k) = \binom{n}{k} \quad \text{(combinations)}$$

Combinations with repetition: $\binom{n+k-1}{k}$

Number of subsets of n elements: 2^n

Number of ways to divide n items into r groups of size

$$n_1, \ldots, n_r : \frac{n!}{n_1! n_2! \ldots n_r!}$$

Inclusion–Exclusion

$$|\cup_{i=1}^{n} A_i| = \sum_{k=1}^{n} (-1)^{k+1} \sum_{1 \le i_1 < \cdots < i_k \le n} |A_{i_1} \cap \cdots \cap A_{i_k}|$$

Stirling / Bell / Catalan

$$S(n,k) = kS(n-1,k) + S(n-1,k-1)$$

(Stirling 2nd kind: partitions of n items into k non-empty sets)

$$B_n = \sum_{k=0}^{n} S(n,k) \quad \text{(Bell number)}$$

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

$$C_0 = 1,\ C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}$$

$$C_n = \frac{(2n)!}{n!(n+1)!}$$

Multinomial theorem

$$(x_1 + x_2 + \cdots + x_m)^n = \sum_{k_1 + \cdots + k_m = n} \frac{n!}{k_1! k_2! \ldots k_m!} x_1^{k_1} x_2^{k_2} \ldots x_m^{k_m}$$

Other useful sums

$$\sum_{k=0}^{n} \binom{r+k}{k} = \binom{r+n+1}{n},$$

$$\sum_{k=0}^{n} (-1)^k \binom{r}{k} = (-1)^n \binom{r-1}{n}$$

$$\sum_{k=0}^{n} \binom{a}{k}\binom{b}{n-k} = \binom{a+b}{n} \quad \text{(Vandermonde's identity)}$$

$$\sum_{k=0}^{n} k\binom{r+k}{k} = (n+1)\binom{r+n+1}{n-1}$$

11.7 matrices

Linear recurrences / Companion matrix

Consider the linear recurrence

$$f_n = \sum_{i=1}^{k} c_i f_{n-i}, \qquad n \ge k,$$

with initial values $f_0, \ldots, f_{k-1}$. Define the $k \times k$ companion matrix T and the $k \times 1$ column vector F_m by

$$T = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_k & c_{k-1} & c_{k-2} & \cdots & c_1 \end{pmatrix}, \qquad F_m = \begin{pmatrix} f_m \\ f_{m+1} \\ \vdots \\ f_{m+k-1} \end{pmatrix}.$$

11.8 vectors

A vector in R^2 is

$$\vec{a} = (a_x, a_y), \quad |\vec{a}| = \sqrt{a_x^2 + a_y^2}, \quad \hat{a} = \frac{\vec{a}}{|\vec{a}|}.$$

Basic Operations

$$\vec{a} \pm \vec{b} = (a_x \pm b_x,\ a_y \pm b_y), \qquad k\vec{a} = (ka_x,\ ka_y)$$

Dot Product

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y = |\vec{a}||\vec{b}| \cos\theta$$

$$\text{proj}_{\vec{b}}(\vec{a}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{b}|^2}\vec{b}, \qquad \vec{a} \cdot \vec{b} = 0 \Rightarrow \text{orthogonal.}$$

2D Cross Product (Scalar Form)

$$\text{cross2D}(\vec{a}, \vec{b}) = a_x b_y - a_y b_x$$

$$|\text{cross2D}(\vec{a}, \vec{b})| = |\vec{a}||\vec{b}| \sin\theta$$

This scalar represents the signed area of the parallelogram formed by $\vec{a}$ and $\vec{b}$.

Areas

$$A_{\text{parallelogram}} = |\text{cross2D}(\vec{a}, \vec{b})|,$$

$$A_{\text{triangle}} = \frac{1}{2}|\text{cross2D}(\vec{a}, \vec{b})|$$

$$A_{\text{polygon}} = \frac{1}{2}\left|\sum_{i=0}^{n-1}(x_i y_{i+1} - y_i x_{i+1})\right|, \quad p_n = p_0.$$

Centroid of a Polygon

$$C_x = \frac{1}{6A}\sum (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i),$$

$$C_y = \frac{1}{6A}\sum (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

Angle and Rotation

Angle between two vectors:

$$\cos\theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|},$$

$$\text{angle}(\vec{a}, \vec{b}) = \text{atan2}(\text{cross2D}(\vec{a}, \vec{b}),\ \text{dot2D}(\vec{a}, \vec{b}))$$

Rotation of a vector by θ:

$$R_\theta(x, y) = (x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta)$$

Rotation about point $P = (p_x, p_y)$:

$$A' = P + R_\theta(A - P)$$

Lines and Intersections

Equation of a line through points A, B:

$$\vec{r}(t) = \vec{A} + t(\vec{B} - \vec{A})$$

Intersection of lines:

$$\vec{L_1} : \vec{r} = \vec{A} + t\vec{v}, \qquad \vec{L_2} : \vec{r} = \vec{B} + s\vec{w}$$

Then

$$t = \frac{\text{cross2D}(B - A,\ \vec{w})}{\text{cross2D}(\vec{v},\ \vec{w})}$$

Distances

Distance from point P to line AB:

$$d = \frac{|\text{cross2D}(\vec{AP},\ \vec{AB})|}{|\vec{AB}|}$$

Projections and Reflections (2D)

Projection of $\vec{a}$ onto unit direction $\hat{b}$:

$$\vec{a}_{\parallel} = (\vec{a} \cdot \hat{b})\hat{b}$$

Perpendicular component:

$$\vec{a}_{\perp} = \vec{a} - \vec{a}_{\parallel}$$

Reflection of $\vec{a}$ about unit normal $\hat{n}$:

$$\vec{a'} = \vec{a} - 2(\vec{a} \cdot \hat{n})\hat{n}$$

Projection of $\vec{a}$ onto line with normal $\hat{n}$:

$$\vec{a'} = \vec{a} - (\vec{a} \cdot \hat{n})\hat{n}$$

12 Z - Templates

12.1 stress

```
#include <bits/stdc++.h>
using namespace std;
const string NAME = "template";
const int NTEST = 100;

mt19937_64
     rd(chrono::steady_clock::now().time_since_epoch().count());
#define rand rd


long long Rand(long long L, long long R) {
    assert(L <= R);
    return L + rd() % (R - L + 1);
}

int main()
{
    srand(time(NULL));
    for (int iTest = 1; iTest <= NTEST; iTest++)
    {
        ofstream inp((NAME + ".inp").c_str());
        // Test gen code here
        inp.close();

        // Change to "./" for Linux
        system((NAME + ".exe").c_str());
        system((NAME + "_trau.exe").c_str());

        // Use diff instead of fc for Linux
        if (system(("fc " + NAME + ".out " + NAME +
              ".ans").c_str()) != 0)
        {
            cout << "Test " << iTest << ": WRONG!\n";
            return 0;
        }
        cout << "Test " << iTest << ": CORRECT!\n";
    }
    return 0;
}
```